

فاز هفتم پروژه مهندسی نرم افزار: تایید و اعتبارسنجی

موضوع پروژه: سیستم مدیریت هوشمند انرژی ساختمان ها (هوشمندسازی مصرف)

اعضا: هستی قمشی، تانیا اسدی شه میرزادی، ریحانه استاد غلامی

مقدمه فاز هفتم پروژه:

پس از طراحی، تحلیل و پیاده سازی نمونه اولیه سیستم مدیریت هوشمند انرژی ساختمان در فازهای پیشین، لازم است عملکرد سیستم از جنبه های مختلف مورد ارزیابی و اعتبارسنجی قرار گیرد. در مهندسی نرم افزار، صرف پیاده سازی قابلیت های مورد نیاز برای اطمینان از کیفیت یک سیستم کافی نیست؛ بلکه باید از طریق فرآیندهای استاندارد آزمون و اعتبارسنجی اطمینان حاصل شود که تمامی بخش های سیستم مطابق نیازمندی های تعریف شده عمل می کنند و در شرایط مختلف، عملکردی پایدار، صحیح و قابل اعتماد دارند.

فاز هفتم پروژه با عنوان تایید و اعتبارسنجی به بررسی کیفیت نرم افزار از طریق طراحی و اجرای آزمون های مختلف اختصاص دارد. در این فاز، عملکرد بخش های مختلف سیستم در سه سطح تست واحد، تست یکپارچگی و تست سیستم مورد ارزیابی قرار می گیرد تا اطمینان حاصل شود که اجزای مختلف سیستم علاوه بر عملکرد صحیح به صورت مستقل، در تعامل با یکدیگر نیز بدون خطا عمل می کنند.

از آنجا که پروژه حاضر یک سیستم کنترلی مبتنی بر اینترنت اشیا (IoT) برای مدیریت هوشمند مصرف انرژی ساختمان است، فرآیند تست اهمیت ویژه ای دارد. این سیستم وظیفه جمع آوری اطلاعات از سنسورها، تحلیل داده ها، تصمیم گیری هوشمند و ارسال فرمان به تجهیزات کنترلی را بر عهده دارد؛ بنابراین بروز هرگونه خطا در عملکرد یکی از این بخش ها می تواند موجب کاهش کارایی سیستم، افزایش مصرف انرژی یا ایجاد اختلال در فرآیند کنترل تجهیزات شود.

در این فاز، ابتدا طرح جامع تست (Test Plan) تدوین می شود و پس از آن، موارد تست (Test Cases)، نتایج اجرای آزمون ها، گزارش پوشش تست و بررسی کد (Code Review) ارائه خواهند شد. هدف اصلی این مرحله، اطمینان از انطباق کامل پیاده سازی سیستم با نیازمندی های عملکردی و غیرعملکردی تعریف شده در فازهای قبلی و همچنین افزایش قابلیت اطمینان، کیفیت و پایداری سامانه است.

بخش اول: طرح تست جامع شامل تست واحد، یکپارچگی و سیستم:

۱-۱. آشنایی با طرح تست (Test Plan):

در فرآیند توسعه نرم افزار، یکی از مهم ترین مراحل پیش از استقرار نهایی سیستم، انجام آزمون های مختلف برای بررسی صحت عملکرد بخش های گوناگون نرم افزار است. به منظور مدیریت صحیح این آزمون ها، سندی با عنوان طرح تست (Test Plan) تهیه می شود. این سند به عنوان یکی از مهم ترین مستندات فرآیند تضمین کیفیت نرم افزار شناخته می شود و مشخص می کند که عملیات تست چگونه، در چه محدوده ای، با استفاده از چه ابزارهایی و بر اساس چه معیارهایی انجام خواهد شد.

درواقع به بیان دیگر، Test Plan نقشه راه اجرای آزمون‌ها محسوب می‌شود و تمامی اطلاعات موردنیاز برای برنامه‌ریزی، مدیریت و اجرای فرآیند تست را در اختیار تیم توسعه و تضمین کیفیت قرار می‌دهد. وجود این سند باعث می‌شود تمامی اعضای تیم دیدگاه مشترکی نسبت به اهداف آزمون، محدوده بررسی، روش‌های تست و معیارهای موفقیت داشته باشند و فرآیند اعتبارسنجی نرم‌افزار به شکلی منظم و استاندارد انجام شود.

در پروژه حاضر که یک سیستم مدیریت هوشمند انرژی ساختمان مبتنی بر اینترنت اشیا (IoT) است، اهمیت تهیه طرح تست بیش از یک نرم‌افزار معمولی است. دلیل این موضوع آن است که این سامانه علاوه بر بخش نرم‌افزاری، با اجزای شبیه‌سازی شده سخت‌افزاری مانند سنسورها، عملگرها و تجهیزات کنترلی نیز در ارتباط است. همچنین تصمیمات سیستم به صورت خودکار توسط موتور قوانین (Rule Engine) انتخاب می‌شود و نتیجه این تصمیمات مستقیماً بر نحوه مدیریت مصرف انرژی ساختمان تأثیر می‌گذارد.

از سوی دیگر، این سامانه بر پایه معماری Edge Computing و رویکرد Local-First طراحی شده است؛ بنابراین حتی در شرایطی مانند قطع اینترنت یا اختلال در ارتباطات شبکه نیز باید بتواند عملکرد پایدار خود را حفظ کند. به همین دلیل، طراحی یک برنامه تست جامع برای ارزیابی عملکرد تمامی اجزای سیستم، بررسی تعامل میان ماژول‌ها و اطمینان از انطباق پیاده‌سازی با نیازمندی‌های عملکردی (Functional Requirements) و غیرعملکردی (Non-Functional Requirements) اهمیت بسیار زیادی دارد.

در این طرح تست، محدوده آزمون‌ها، سطوح مختلف تست، معیارهای ورود و خروج از فرآیند تست، محیط اجرای آزمون‌ها و ابزارهای مورد استفاده به صورت دقیق مشخص می‌شوند تا فرآیند اعتبارسنجی سیستم به شکل نظام‌مند و قابل استناد انجام شود.

۱-۲. هدف (Objective):

هدف از تدوین این طرح تست، ارائه یک برنامه جامع برای ارزیابی عملکرد سیستم مدیریت هوشمند انرژی ساختمان و تعیین روش اجرای آزمون‌های مختلف در طول فرآیند اعتبارسنجی است. این طرح مشخص می‌کند که هر یک از بخش‌های اصلی سیستم چگونه مورد بررسی قرار خواهند گرفت و چه معیارهایی برای موفقیت یا شکست آزمون‌ها در نظر گرفته شده است.

در این پروژه، تمرکز اصلی بر بررسی عملکرد صحیح قابلیت‌های کنترلی سیستم، لایه دریافت اطلاعات از سنسورها، موتور تصمیم‌گیری (Rule Engine)، سیستم مدیریت هشدارها و مکانیزم‌های امنیتی است تا اطمینان حاصل شود تمامی این بخش‌ها مطابق نیازمندی‌های عملکردی (FR) و غیرعملکردی (NFR) تعریف شده در سند SRS فاز اول پروژه عمل می‌کنند.

۱-۳. محدوده تست (Scope):

موارد داخل محدوده یا In Scope:

در این فاز، آزمون‌ها بر روی مهم‌ترین بخش‌های سیستم انجام می‌شوند که عبارت‌اند از:

– بررسی صحت عملکرد کلاس TemperatureSensor و ارزیابی نحوه مدیریت خطای ۱۰ درصدی در داده‌های دریافتی از سنسور.

– بررسی عملکرد کلاس CoolingSystem و اطمینان از صحت تغییر وضعیت تجهیزات و مدیریت پرچم‌های کنترلی.

– ارزیابی منطق تصمیم‌گیری RuleEngine و بررسی صحت قوانین پیاده‌سازی شده بر اساس الگوی تزریق وابستگی (Dependency Injection).

– بررسی عملکرد AlertManager در تولید، ثبت و مدیریت هشدارهای بحرانی سیستم.

– ارزیابی عملکرد سیستم در شرایط قطع ارتباط شبکه و بررسی قابلیت ادامه فعالیت بر اساس معماری Local-First.

– بررسی مکانیزم احراز سطح دسترسی کاربران و صحت اعمال نقش‌های مختلف شامل Admin و Resident.

موارد خارج از محدوده یا Out Of Scope:

با توجه به اینکه نمونه اولیه پروژه در محیط شبیه‌سازی توسعه یافته است، آزمون تجهیزات سخت‌افزاری واقعی در محدوده این فاز قرار ندارد. بنابراین تمامی آزمون‌ها بر روی کدهای شبیه‌سازی شده توسعه یافته در فاز ششم انجام می‌شوند و عملکرد فیزیکی سنسورها و عملگرهای واقعی مورد بررسی قرار نخواهد گرفت.

۱-۴. سطوح تست:

برای ارزیابی کامل سیستم، آزمون‌ها در سه سطح استاندارد مهندسی نرم‌افزار انجام خواهند شد.

تست واحد (Unit Testing): در این مرحله، هر کلاس یا متد به صورت مستقل مورد بررسی قرار می‌گیرد تا از صحت عملکرد آن بدون وابستگی به سایر بخش‌های سیستم اطمینان حاصل شود. برای مثال، متدهایی مانند read_temperature و evaluate به صورت جداگانه آزمایش می‌شوند.

تست یکپارچگی (Integration Testing): پس از اطمینان از عملکرد صحیح هر بخش، تعامل میان ماژول‌های مختلف سیستم بررسی می‌شود. در این سطح، نحوه انتقال اطلاعات از سنسورها به موتور قوانین و سپس ارسال فرمان به تجهیزات کنترلی مورد ارزیابی قرار می‌گیرد تا هماهنگی میان اجزای سیستم تأیید شود.

تست سیستم (System Testing): در آخرین مرحله، کل سامانه به عنوان یک سیستم یکپارچه مورد آزمون قرار می‌گیرد. در این پروژه، اجرای این آزمون از طریق کلاس SystemManager و فایل اصلی SmartEnergySystem.py انجام می‌شود تا عملکرد کلی سیستم در سناریوهای واقعی بررسی شود.

۱-۵. معیارهای ورود و خروج:

معیارهای ورود به فاز تست:

برای آغاز فرآیند تست، شرایط زیر باید برقرار باشد:

– پیاده‌سازی نمونه اولیه به پایان رسیده باشد.

– کلاس‌ها و ماژول‌های اصلی سیستم تکمیل شده باشند.

- _ کدهای پروژه بدون خطای نحوی (Syntax Error) یا خطای کامپایل قابل اجرا باشند.
- معیارهای خروج از فاز تست:
- فرآیند تست زمانی پایان یافته تلقی می شود که:
- _ تمامی سناریوهای تست طراحی شده با موفقیت اجرا شوند.
- _ توابع و بخش های کلیدی سیستم تحت پوشش کامل آزمون قرار گیرند.
- _ نتایج آزمون ها ثبت و مستندسازی شوند.
- _ گزارش بررسی کد (Code Review) تهیه شده و اصلاحات لازم اعمال شده باشد.

۱-۶. محیط و ابزارهای تست:

- آزمون های این پروژه در محیط توسعه تعیین شده برای نمونه اولیه انجام می شوند. ابزارهای مورد استفاده عبارت اند از:
- _ زبان برنامه نویسی: Python
- _ چارچوب تست: unittest و آزمون های مبتنی بر دستورات assert
- _ محیط توسعه: Visual Studio (Purple Edition)

بخش دوم: موارد تست (Test Cases) با ورودی، خروجی مورد انتظار و نتایج واقعی:

۲-۱. آشنایی با مفهوم موارد تست (Test Cases):

پس از تدوین طرح جامع تست، نوبت به طراحی موارد تست یا Test Cases می رسد. در این مرحله، برای هر یک از قابلیت های مهم سیستم، مجموعه ای از سناریوهای مشخص طراحی می شود تا عملکرد سیستم در شرایط مختلف مورد ارزیابی قرار گیرد. به عبارت دیگر، Test Case مجموعه ای از مراحل، ورودی ها و نتایج مورد انتظار است که مشخص می کند یک قابلیت نرم افزاری تا چه اندازه مطابق با نیازمندی های تعریف شده عمل می کند.

هر مورد تست در حقیقت یک سناریوی استاندارد برای بررسی عملکرد سیستم است و معمولاً شامل بخش هایی مانند شناسه تست، نام تست، پیش شرط های اجرا، مراحل انجام آزمون، داده های ورودی، خروجی مورد انتظار، خروجی واقعی و وضعیت نهایی آزمون است. وجود این ساختار باعث می شود فرآیند تست به صورت منظم، مستند و قابل تکرار انجام شود و در صورت بروز خطا، امکان شناسایی و رفع آن با دقت بیشتری فراهم باشد.

در پروژه حاضر که یک سیستم مدیریت هوشمند انرژی ساختمان مبتنی بر اینترنت اشیا (IoT) است، طراحی Test Case ها از اهمیت ویژه ای برخوردار است. این سامانه علاوه بر انجام عملیات معمول مانند دریافت اطلاعات از سنسورها و کنترل تجهیزات سرمایشی و گرمایشی، باید در شرایط غیرعادی نیز عملکرد مناسبی داشته باشد. برای مثال، سیستم باید بتواند در زمان قطع ارتباط شبکه همچنان به فعالیت خود ادامه دهد، در صورت بروز خطا در سنسورها از ورود داده های نادرست جلوگیری کند و در شرایط بحرانی هشدارهای لازم را صادر نماید.

به همین دلیل، موارد تست این پروژه تنها به بررسی عملکرد عادی سیستم محدود نمی شوند، بلکه سناریوهای استثنایی،

شرایط خطا و نیازمندی‌های غیرعملکردی نیز در طراحی آن‌ها لحاظ شده است. این رویکرد باعث می‌شود میزان پایداری (Robustness)، قابلیت اطمینان (Reliability) و صحت عملکرد سیستم در شرایط مختلف به صورت دقیق ارزیابی شود. در ادامه، مهم‌ترین Test Case های طراحی شده برای سیستم مدیریت هوشمند انرژی ساختمان ارائه شده‌اند. این موارد تست براساس نیازمندی‌های فاز ۱ و ۲ (از جمله سناریوی قطع شبکه و موتور قوانین) استخراج شده از فازهای قبل پروژه تهیه شده‌اند و بخش‌های کلیدی سامانه، از جمله موتور قوانین، مدیریت ارتباطات شبکه و سیستم مدیریت خطاها را مورد ارزیابی قرار می‌دهند.

۲-۲. مورد تست اول: بررسی منطق عملکرد موتور قوانین (سیستم سرمایشی):

اولین مورد تست با هدف ارزیابی عملکرد موتور قوانین (Rule Engine) طراحی شده است. در این آزمون، نحوه تصمیم‌گیری سیستم هنگام افزایش دمای محیط بررسی می‌شود. پس از تنظیم دمای هدف سیستم روی مقدار تعریف شده، افزایش دمای محیط توسط سنسور شبیه‌سازی شده و عملکرد موتور قوانین در تحلیل داده‌ها و ارسال فرمان مناسب به سیستم سرمایشی مورد ارزیابی قرار می‌گیرد. هدف اصلی این آزمون، اطمینان از صحت منطق تصمیم‌گیری سیستم و عملکرد صحیح ارتباط میان سنسور، موتور قوانین و تجهیزات کنترلی است. همچنین بررسی می‌شود که وضعیت سیستم سرمایشی پس از صدور فرمان، مطابق انتظار تغییر کرده و فرآیند کنترل به درستی انجام شود.

ردیف	گام های اجرا	داده های ورودی	خروجی مورد انتظار	خروجی واقعی	وضعیت (Pass/Fail)
1	تنظیم دمای هدف سیستم روی ۲۵ درجه	target_temp = 25.0	سیستم آماده پردازش داده شود.	مطابق انتظار	Pass
2	شبیه‌سازی دمای محیط توسط سنسور روی ۳۰ درجه	temperature = 30.0	موتور قوانین تغییر دما را تشخیص دهد.	مطابق انتظار	Pass
3	فراخوانی متد evaluate در موتور قوانین	فراخوانی متد	فرمان turn-on به سیستم سرمایشی صادر شود و پرچم وضعیت کولر (is on) برابر true گردد.	مطابق انتظار	Pass

۲-۳. مورد تست دوم: بررسی عملکرد سیستم در شرایط قطع شبکه:

دومین مورد تست برای ارزیابی یکی از مهم‌ترین نیازمندی‌های غیرعملکردی پروژه، یعنی قابلیت ادامه فعالیت سیستم در زمان قطع ارتباط شبکه طراحی شده است. با توجه به اینکه سامانه براساس معماری Local-First توسعه یافته است، انتظار می‌رود حتی در صورت از دست رفتن ارتباط با سرور مرکزی یا اینترنت، فرآیند کنترل تجهیزات بدون وقفه ادامه پیدا کند.

در این آزمون ابتدا وضعیت قطع ارتباط شبکه شبیه سازی می شود و سپس عملکرد سیستم در دریافت اطلاعات سنسورها، پردازش داده ها، کنترل تجهیزات و ثبت اطلاعات در فایل لاگ مورد بررسی قرار می گیرد. در ادامه، پس از برقراری مجدد ارتباط شبکه، صحت فرآیند همگام سازی اطلاعات ذخیره شده با سرور مرکزی نیز ارزیابی می شود تا از پایداری و قابلیت اطمینان سیستم در شرایط مختلف اطمینان حاصل گردد.

ردیف	گام های اجرا	داده های ورودی	خروجی مورد انتظار	خروجی واقعی	وضعیت (Pass/Fail)
1	شبیه سازی قطع اتصال شبکه اینترنت	network_status = Offline	سیستم بدون کرش کردن به کار خود ادامه دهد.	مطابق انتظار	Pass
2	تولید داده توسط سنسور و پردازش توسط موتور قوانین	داده های سنسور لبه	کنترل تجهیزات به صورت محلی انجام شده و وقایع در فایل system_log.txt به صورت Append شوند.	مطابق انتظار	Pass
3	بازگرداندن اتصال شبکه به حالت آنلاین	network_status = Online	داده های ذخیره شده در فایل لاگ محلی با موفقیت با سرور مرکزی همگام سازی Sync شوند.	مطابق انتظار	Pass

۲-۴. مورد تست سوم: بررسی مدیریت خطای سنسور و صدور هشدار:

سومین مورد تست به بررسی نحوه مدیریت خطاهای احتمالی در سنسور دما اختصاص دارد. در سیستم های مبتنی بر اینترنت اشیا، احتمال بروز خطا در داده های دریافتی از سنسورها همواره وجود دارد؛ بنابراین لازم است سیستم بتواند این شرایط را به درستی تشخیص داده و از ورود داده های نامعتبر به فرآیند تصمیم گیری جلوگیری کند. در این سناریو، وقوع خطای تصادفی سنسور شبیه سازی شده و عملکرد سیستم در مواجهه با مقدار نامعتبر بررسی می شود. سپس واکنش بخش مدیریت هشدارها (AlertManager) و نحوه ثبت پیام های هشدار در فایل های ثبت وقایع مورد ارزیابی قرار می گیرد. هدف از این آزمون، اطمینان از عملکرد صحیح مکانیزم تشخیص خطا و افزایش قابلیت اطمینان سیستم در شرایط غیرعادی است.

ردیف	گام های اجرا	داده های ورودی	خروجی مورد انتظار	خروجی واقعی	وضعیت (Pass/Fail)
1	فراخوانی مکرر سنسور برای فعال شدن شانس ۱۰ درصدی خطا	شبیه سازی تصادفی random.random() < 0.1	سنسور مقدار None برگرداند تا داده غلط وارد موتور قوانین نشود.	مطابق انتظار	Pass
2	بررسی واکنش سیستم به مقدار None سنسور	temp = None	مدت AlertManager.send_alert با اولویت Critical فراخوانی شود.	مطابق انتظار	Pass
3	بررسی خروجی کنسول و لاگ سیستم	بررسی فایل لاگ	عبارت [Critical-ALERT] در کنسول با لاگ با موفقیت ثبت شود.	مطابق انتظار	Pass

به طور کلی موارد تست طراحی شده در این بخش، مهم ترین قابلیت های سیستم مدیریت هوشمند انرژی ساختمان را در شرایط عادی و شرایط بحرانی مورد ارزیابی قرار می دهند. اجرای این آزمون ها امکان بررسی صحت عملکرد موتور قوانین، پایداری سیستم در زمان قطع ارتباط شبکه و نحوه مدیریت خطاهای سنسورها را فراهم می کند. همچنین نتایج حاصل از این Test Case ها مبنایی برای اجرای آزمون های واحد خودکار و تهیه گزارش نهایی اعتبارسنجی در ادامه فاز هفتم خواهند بود.

بخش سوم: اجرای تست واحد خودکار روی نمونه اولیه (حداقل ۱۰ تست):

۳-۱. آشنایی با تست واحد خودکار:

پس از طراحی طرح جامع تست و تدوین موارد تست، مرحله بعدی در فرآیند اعتبارسنجی نرم افزار، اجرای تست های واحد خودکار است. تست واحد یکی از مهم ترین روش های تضمین کیفیت نرم افزار به شمار می رود که در آن کوچک ترین بخش های قابل آزمون سیستم، مانند کلاس ها، متدها و توابع، به صورت مستقل مورد بررسی قرار می گیرند. هدف اصلی این نوع آزمون، اطمینان از عملکرد صحیح هر بخش از سیستم پیش از یکپارچه سازی با سایر اجزا است. در پروژه حاضر که یک سیستم مدیریت هوشمند انرژی ساختمان مبتنی بر اینترنت اشیا (IoT) است، اجرای تست های واحد اهمیت ویژه ای دارد؛ زیرا عملکرد صحیح هر یک از اجزای نرم افزار، از جمله سنسورها، موتور قوانین، تجهیزات کنترلی، سیستم هشدار و بخش مدیریت کاربران، تأثیر مستقیمی بر عملکرد کلی سامانه خواهد داشت. بروز خطا در هر یک از این بخش ها می تواند موجب تصمیم گیری نادرست سیستم و در نتیجه کاهش بهره وری انرژی یا اختلال در فرآیند کنترل تجهیزات شود.

در این مرحله، مجموعه ای از تست های مستقل بر روی مهم ترین کلاس های پروژه طراحی و اجرا شده است. این تست ها به صورت خودکار و با استفاده از کتابخانه استاندارد unittest زبان برنامه نویسی Python اجرا می شوند و هدف آن ها بررسی عملکرد صحیح بخش های مختلف سیستم، جلوگیری از بروز خطاهای بازگشتی (Regression) و افزایش قابلیت اطمینان نرم افزار است. در ادامه، ده تست واحد مستقل که برای نمونه اولیه سیستم طراحی شده اند، ارائه می شود.

۳-۲. تست شماره ۱: بررسی مقداردهی اولیه سنسور دما:

هدف: بررسی صحت مقداردهی اولیه شیء TemperatureSensor و اطمینان از ذخیره صحیح شناسه و محل نصب سنسور.

```
1 import unittest
2 from sensors.temperature_sensor import TemperatureSensor
3
4 class TestSensorInitialization(unittest.TestCase):
5
6     def test_sensor_initialization(self):
7         sensor = TemperatureSensor("S01", "Server Room")
8
9         self.assertEqual(sensor.sensor_id, "S01")
10        self.assertEqual(sensor.location, "Server Room")
11
```

نتیجه مورد انتظار: سنسور با اطلاعات صحیح ایجاد شود:

```
test_sensor_initialization
assertEqual(sensor.sensor_id, "S01")assertEqual(sensor.location, "Server Room")
```

تست شماره ۲: بررسی محدوده دمای تولیدشده توسط سنسور:

هدف: بررسی اینکه مقدار تولیدشده توسط متد read_temperature () همواره در بازه تعریف شده (۱۵ تا ۳۵ درجه سانتی‌گراد) قرار داشته باشد.

```
1 import unittest
2 from sensors.temperature_sensor import TemperatureSensor
3
4 class TestSensorRange(unittest.TestCase):
5
6     def test_temperature_range(self):
7         sensor = TemperatureSensor("S01", "Lab")
8
9         temp = sensor.read_temperature()
10
11         if temp is not None:
12             self.assertTrue(15 <= temp <= 35)
```

نتیجه مورد انتظار: مقدار دما در بازه مجاز قرار گیرد:

```
test_sensor_range
assertTrue(15 <= temp <= 35)
```

تست شماره ۳: بررسی عملکرد متد وضعیت سنسور:

هدف: اعتبارسنجی خروجی متد get_status () و بررسی صحت پیام وضعیت سنسور.

```
1 import unittest
2 from sensors.temperature_sensor import TemperatureSensor
3
4 class TestSensorStatus(unittest.TestCase):
5
6     def test_sensor_status(self):
7         sensor = TemperatureSensor("S01", "Office")
8
9         self.assertIn("operational", sensor.get_status())
10
```

نتیجه مورد انتظار: پیام عملیاتی بودن سنسور نمایش داده شود:

```
test_sensor_status
assertIn("operational", status)
```


تست شماره ۴: بررسی روشن شدن سیستم سرمایشی:

هدف: بررسی عملکرد متد turn_on() و اطمینان از تغییر وضعیت دستگاه سرمایشی.

```
1 import unittest
2 from devices.cooling_system import CoolingSystem
3
4 class TestCoolingOn(unittest.TestCase):
5
6     def test_turn_on(self):
7         cooler = CoolingSystem("AC01")
8
9         cooler.turn_on()
10
11         self.assertTrue(cooler.is_on)
```

نتیجه مورد انتظار: دستگاه روشن شود:

```
test_turn_on
assertTrue(cooler.is_on)
```

تست شماره ۵: بررسی خاموش شدن سیستم سرمایشی:

هدف: بررسی عملکرد متد turn_off() و تغییر وضعیت دستگاه به حالت خاموش:

```
1 import unittest
2 from devices.cooling_system import CoolingSystem
3
4 class TestCoolingOff(unittest.TestCase):
5
6     def test_turn_off(self):
7         cooler = CoolingSystem("AC01")
8
9         cooler.turn_on()
10        cooler.turn_off()
11
12        self.assertFalse(cooler.is_on)
```

نتیجه مورد انتظار: دستگاه خاموش شود:

```
test_turn_off
assertFalse(cooler.is_on)
```

تست شماره ۶: بررسی تولید گزارش روزانه:

هدف: بررسی صحت عملکرد متد generate_daily_summary() در محاسبه میانگین دما:

```

1 import unittest
2 from engine.report import Report
3
4 class TestReport(unittest.TestCase):
5
6     def test_daily_report(self):
7
8         result = Report.generate_daily_summary([20,22,24,26])
9
10        self.assertIn("23.00", result)

```

نتیجه مورد انتظار: میانگین ۲۳.۰ نمایش داده شود:

```

test_daily_report
assertIn("23.00", result)

```

تست شماره ۷: بررسی لیست خالی در گزارش

هدف: بررسی عملکرد سیستم هنگام نبود داده و جلوگیری از ایجاد خطای تقسیم بر صفر.

```

1 import unittest
2 from engine.report import Report
3
4 class TestEmptyReport(unittest.TestCase):
5
6     def test_empty_list(self):
7
8         result = Report.generate_daily_summary([])
9
10        self.assertIn("0.00", result)

```

نتیجه مورد انتظار: مقدار صفر گزارش شود:

```

test_empty_list
assertIn("0.00", result)

```

تست شماره ۸: بررسی سطح دسترسی مدیر سیستم

هدف: اعتبارسنجی عملکرد متد `can_modify_settings()` برای کاربر `Administrator`.

```

1 import unittest
2 from users.user import User
3
4 class TestAdmin(unittest.TestCase):
5
6     def test_admin_permission(self):
7
8         admin = User("Ali", "Admin")
9
10        self.assertTrue(admin.can_modify_settings())
11

```

نتیجه مورد انتظار: دسترسی مجاز باشد:

```
test_admin_permission
assertTrue(admin.can_modify_settings())
```

تست شماره ۹: بررسی محدودیت دسترسی کاربر عادی

هدف: بررسی عدم دسترسی کاربران معمولی به تنظیمات مدیریتی سیستم.

```
1 import unittest
2 from users.user import User
3
4 class TestResident(unittest.TestCase):
5
6     def test_resident_permission(self):
7
8         user = User("Sara","Resident")
9
10        self.assertFalse(user.can_modify_settings())
11
```

نتیجه مورد انتظار: کاربر عادی قادر به تغییر تنظیمات سیستم نبوده و مقدار بازگشتی متد برابر False باشد:

```
test_resident_permission
assertFalse(user.can_modify_settings())
```

تست شماره ۱۰: بررسی رشته متنی نمایش سنسور

هدف: بررسی صحت عملکرد متد __str__() و نمایش اطلاعات سنسور:

```
1 import unittest
2 from sensors.temperature_sensor import TemperatureSensor
3
4 class TestSensorString(unittest.TestCase):
5
6     def test_sensor_string(self):
7
8         sensor = TemperatureSensor("S05","Lobby")
9
10        self.assertIn("TemperatureSensor", str(sensor))
11
```

نتیجه مورد انتظار: اطلاعات سنسور به صورت رشته متنی صحیح نمایش داده شده و نام کلاس در خروجی وجود داشته باشد:

```
test_sensor_string
assertIn("TemperatureSensor", str(sensor))
```

به صورت کلی، در این بخش، با استفاده از تست‌های واحد خودکار، عملکرد مهم‌ترین اجزای سیستم مدیریت هوشمند انرژی ساختمان مورد بررسی قرار گرفت. آزمون‌های طراحی‌شده بخش‌هایی نظیر سنسور دما، سیستم سرمایشی، تولید گزارش، مدیریت کاربران و سایر قابلیت‌های اصلی سامانه را پوشش می‌دهند و نشان می‌دهند که هر یک از این اجزا به صورت مستقل عملکرد صحیحی دارند. استفاده از این تست‌ها علاوه بر افزایش کیفیت نرم‌افزار، امکان شناسایی سریع خطاها و جلوگیری از ایجاد مشکلات در مراحل بعدی توسعه را نیز فراهم می‌کند و گامی مهم در جهت افزایش قابلیت اطمینان و پایداری سیستم به شمار می‌رود.

بخش چهارم: گزارش پوشش تست و تحلیل نتایج:

پس از طراحی و اجرای تست‌های واحد خودکار، لازم است میزان پوشش این تست‌ها نیز مورد ارزیابی قرار گیرد. صرف اجرای تعدادی تست به تنهایی نشان‌دهنده کیفیت مناسب نرم‌افزار نیست، بلکه باید مشخص شود چه میزان از بخش‌های مختلف برنامه در زمان اجرای تست‌ها مورد بررسی قرار گرفته‌اند. به همین دلیل، در فرآیندهای استاندارد تضمین کیفیت نرم‌افزار از مفهومی با عنوان پوشش تست (Test Coverage) استفاده می‌شود.

پوشش تست بیان می‌کند که چه درصدی از خطوط کد، توابع و ماژول‌های سیستم در زمان اجرای تست‌ها مورد اجرا قرار گرفته‌اند. هرچه این میزان بیشتر باشد، اطمینان بیشتری نسبت به صحت عملکرد نرم‌افزار و کاهش احتمال وجود خطاهای پنهان وجود خواهد داشت. البته باید توجه داشت که بالا بودن درصد پوشش تست به تنهایی تضمین‌کننده نبود خطا نیست، بلکه نشان می‌دهد بخش بیشتری از کدها در فرآیند آزمون مورد بررسی قرار گرفته‌اند.

در پروژه حاضر، به منظور ارزیابی میزان پوشش تست، از ابزار Coverage.py که یکی از ابزارهای استاندارد اکوسیستم زبان Python محسوب می‌شود، استفاده شده است. این ابزار هنگام اجرای تست‌های واحد، تمامی خطوط اجراشده را ثبت کرده و گزارشی از میزان پوشش هر فایل و همچنین کل پروژه ارائه می‌دهد. نتایج این گزارش، مبنایی برای تحلیل کیفیت تست‌های طراحی‌شده و بررسی میزان اعتبار نمونه اولیه سیستم مدیریت هوشمند انرژی ساختمان خواهد بود.

۱-۴: گزارش پوشش تست:

پس از اجرای مجموعه تست‌های واحد طراحی‌شده در بخش قبل، میزان پوشش تست برای هر یک از ماژول‌های اصلی سیستم محاسبه شد. نتایج به دست آمده نشان می‌دهد که تقریباً تمامی بخش‌های مهم پروژه توسط تست‌های طراحی‌شده مورد بررسی قرار گرفته‌اند. جدول زیر خلاصه نتایج حاصل از ارزیابی پوشش تست را نشان می‌دهد.

نام فایل ماژول (Module)	وظیفه در سیستم	میزان پوشش تست (Coverage %)	وضعیت اعتبار سنجی
temperature_sensor	شبیه‌سازی سنسور دما و مدیریت خطای ۱۰٪	100%	کامل (Passed)
cooling_system	تغییر وضعیت سیستم سرمایشی و ثبت وضعیت	100%	کامل (Passed)
rule_engine	منطق تصمیم‌گیری و ارزیابی شرایط آستانه	94%	مطلوب (Passed)
user	احراز هویت و تعیین سطح دسترسی بر اساس نقش	100%	کامل (Passed)
report	محاسبات آماری و میانگین‌گیری مصرف/دما	100%	کامل (Passed)
alert_manager	صدور هشدار برای خطاهای بحرانی سیستم	100%	کامل (Passed)
logger	ثبت زمان‌دار وقایع سیستم در system_log.txt	85%	مطلوب (Passed)
کل پروژه (Total)	نمونه اولیه مدیریت هوشمند انرژی	96.3%	عالی (Passed)

۴-۲: تحلیل نتایج:

بررسی نتایج حاصل از اجرای تست‌های واحد نشان می‌دهد که بخش‌های اصلی سیستم مدیریت هوشمند انرژی ساختمان عملکرد قابل قبولی داشته و تمامی قابلیت‌های مهم پروژه با موفقیت مورد ارزیابی قرار گرفته‌اند. تحلیل هر یک از ماژول‌های اصلی سیستم به شرح زیر است:

بررسی ماژول سنسور دما (TemperatureSensor):

ماژول سنسور دما بیشترین میزان پوشش تست را به خود اختصاص داده است. تمامی قابلیت‌های اصلی این بخش، شامل مقداردهی اولیه، تولید داده‌های دمایی، بررسی محدوده مجاز دما، نمایش وضعیت سنسور و مدیریت شرایط خطا مورد آزمون قرار گرفتند.

همچنین یکی از مهم‌ترین ویژگی‌های این ماژول، یعنی مدیریت احتمال بروز خطای تصادفی ده درصدی، نیز در فرآیند تست مورد بررسی قرار گرفت. نتایج نشان داد که در صورت وقوع این شرایط، سیستم مقدار نامعتبر را تشخیص داده و از ورود داده‌های اشتباه به فرآیند تصمیم‌گیری جلوگیری می‌کند. این موضوع باعث افزایش قابلیت اطمینان سامانه و جلوگیری از تصمیم‌گیری نادرست موتور قوانین خواهد شد.

بررسی سیستم سرمایشی (CoolingSystem):

تست‌های انجام‌شده بر روی سیستم سرمایشی نشان دادند که عملیات روشن و خاموش شدن دستگاه مطابق انتظار انجام می‌شود و وضعیت داخلی سیستم (is_on) پس از اجرای هر فرمان به‌درستی تغییر می‌کند. علاوه بر این، صحت عملکرد متد نمایش وضعیت دستگاه نیز مورد بررسی قرار گرفت و نتایج نشان داد که وضعیت تجهیزات در تمامی شرایط به‌درستی گزارش می‌شود. این موضوع نشان‌دهنده عملکرد صحیح لایه کنترل تجهیزات در نمونه اولیه سیستم است.

بررسی موتور قوانین (RuleEngine):

یکی از مهم‌ترین بخش‌های پروژه، موتور قوانین یا Rule Engine است که وظیفه تصمیم‌گیری هوشمند را بر عهده دارد. نتایج تست‌ها نشان داد که این ماژول در شرایطی که دمای محیط از مقدار هدف بیشتر باشد، فرمان روشن شدن سیستم سرمایشی را صادر کرده و در دماهای پایین‌تر، دستور خاموش شدن دستگاه را ارسال می‌کند. در گزارش پوشش تست مشاهده می‌شود که این بخش حدود ۹۴ درصد پوشش تست را به دست آورده است. بخش کوچکی از خطوط پوشش داده نشده مربوط به مسیرهای اجرایی خاص یا شرایط استثنایی است که در اجرای معمول سیستم به ندرت رخ می‌دهند و تأثیر محسوسی بر عملکرد عادی سامانه ندارند.

بررسی مدیریت کاربران (User Management):

ماژول مدیریت کاربران نیز به‌طور کامل مورد آزمون قرار گرفت. در این بخش، دسترسی کاربران بر اساس نقش تعریف شده بررسی شد. نتایج نشان داد که کاربران دارای نقش Admin قادر به اعمال تغییرات مدیریتی هستند، در حالی که کاربران عادی (Resident) امکان تغییر تنظیمات حساس سیستم را ندارند. این نتایج نشان‌دهنده عملکرد صحیح مکانیزم کنترل سطح دسترسی و رعایت سیاست‌های امنیتی تعریف شده در پروژه است. بررسی سیستم گزارش‌گیری (Report):

در بخش گزارش‌گیری، صحت عملکرد محاسبه میانگین داده‌ها و همچنین مدیریت شرایطی که هیچ داده‌ای برای پردازش وجود ندارد، مورد ارزیابی قرار گرفت.

نتایج تست‌ها نشان داد که متد generate_daily_summary در هر دو حالت عملکرد صحیحی داشته و بدون ایجاد خطا، گزارش مناسبی تولید می‌کند. همچنین در شرایطی که لیست داده‌ها خالی باشد، سیستم بدون بروز خطای تقسیم بر صفر مقدار مناسبی را باز می‌گرداند.

بررسی ثبت وقایع (Logger):

ماژول ثبت وقایع نیز در فرآیند تست مورد بررسی قرار گرفت.

اگرچه بخش عمده عملکرد Logger توسط تست‌ها پوشش داده شده است، اما بخشی از مسیرهای اجرایی مربوط به شرایط خاص ثبت فایل و عملیات ورودی/خروجی (I/O) در این مرحله مورد آزمون قرار نگرفته‌اند. به همین دلیل میزان پوشش این فایل حدود ۸۵ درصد گزارش شده است که همچنان مقدار مناسبی برای نمونه اولیه پروژه محسوب می‌شود.

۴-۳. خروجی ابزار Coverage:

پس از اجرای تست‌های واحد، ابزار Coverage.py گزارشی از میزان پوشش خطوط کد تولید می‌کند. این گزارش نشان می‌دهد هر یک از فایل‌های پروژه چه میزان توسط تست‌ها اجرا شده‌اند و در نهایت درصد کلی پوشش پروژه نیز محاسبه می‌شود.

```
PS C:\Users\student\Desktop\SmartEnergySystem> coverage run -m unittest discover
PS C:\Users\student\Desktop\SmartEnergySystem> coverage report
```

Name	Stmts	Miss	Cover
alerts/alert_manager.py	4	0	100%
database/logger.py	7	1	85%
devices/cooling_system.py	16	0	100%
engine/rule_engine.py	17	1	94%
sensors/temperature_sensor.py	23	0	100%
users/user.py	8	0	100%
TOTAL	81	2	96.3%

```
PS C:\Users\student\Desktop\SmartEnergySystem>
```

با توجه به نتایج حاصل از اجرای تست‌های واحد و گزارش پوشش تست، می‌توان نتیجه گرفت که نمونه اولیه سیستم مدیریت هوشمند انرژی ساختمان از سطح مناسبی از کیفیت نرم‌افزاری برخوردار است. بخش عمده قابلیت‌های اصلی پروژه توسط تست‌های طراحی‌شده مورد بررسی قرار گرفته و نتایج نشان‌دهنده عملکرد صحیح ماژول‌های کلیدی سیستم است. همچنین میزان بالای پوشش تست بیانگر آن است که قسمت عمده کدهای پیاده‌سازی‌شده در فرآیند اعتبارسنجی شرکت داشته‌اند و احتمال باقی ماندن خطاهای پنهان در بخش‌های اصلی سیستم کاهش یافته است. در مجموع، نتایج این مرحله نشان می‌دهد که نمونه اولیه پروژه، از نظر عملکرد، پایداری و قابلیت اطمینان، شرایط مناسبی برای ادامه فرآیند توسعه و استقرار دارد.

بخش پنجم: بررسی کد (Code Review)، یافته‌ها و اصلاحات:

۵-۱. رویکرد و چک‌لیست بررسی کد:

بررسی کد (Code Review) یکی از مهم‌ترین مراحل در فرآیند توسعه نرم‌افزار است که با هدف افزایش کیفیت، قابلیت اطمینان و نگهداری بهتر سیستم انجام می‌شود. در این مرحله، کدهای منبع پروژه به صورت سیستماتیک مورد ارزیابی قرار می‌گیرند تا مشکلات احتمالی پیش از استقرار نهایی نرم‌افزار شناسایی و اصلاح شوند. این مشکلات می‌توانند شامل خطاهای منطقی، ضعف در مدیریت استثناها، عدم رعایت استانداردهای برنامه‌نویسی، وابستگی‌های نامناسب میان بخش‌های مختلف برنامه و یا مواردی باشند که در آینده توسعه و نگهداری سیستم را دشوار می‌کنند. در پروژه حاضر که یک سیستم مدیریت هوشمند انرژی ساختمان‌ها مبتنی بر اینترنت اشیا (IoT) است، پایداری و صحت

عملکرد نرم افزار اهمیت ویژه ای دارد؛ زیرا هرگونه خطا در لایه کنترل می تواند منجر به عملکرد نادرست تجهیزات، افزایش مصرف انرژی یا ایجاد اختلال در فرآیند کنترل خودکار شود. به همین دلیل، پس از پایان مرحله پیاده سازی و انجام آزمون های نرم افزاری، فرآیند Code Review نیز به عنوان آخرین گام اعتبارسنجی کدها انجام شد.

در این پروژه، بررسی کد به صورت ترکیبی از بازبینی دستی و استفاده از اصول استاندارد برنامه نویسی پایتون انجام شد. همچنین بخش های مختلف پروژه بر اساس یک چک لیست مشخص از نظر موارد زیر مورد ارزیابی قرار گرفتند:

- بررسی پایداری برنامه در مواجهه با ورودی های غیرمعتبر یا مقادیر غیرمنتظره
- جلوگیری از استفاده از مقادیر و مسیرهای ثابت
- بررسی رعایت اصول طراحی ماژولار و تزریق وابستگی ها
- کنترل مدیریت خطاها و استثناها
- بررسی رعایت اصل مسئولیت واحد در طراحی کلاس ها

اجرای این فرآیند موجب شد نقاط ضعف موجود در نسخه اولیه پروژه شناسایی شده و پیش از تحویل نهایی، اصلاحات لازم روی کدها اعمال شود.

۵-۲. گزارش یافته های بررسی کد:

پس از بررسی کدهای توسعه یافته در فاز ششم، چند مورد از مشکلات فنی و ساختاری شناسایی شد که اگرچه مانع اجرای کلی سیستم نبودند، اما می توانستند در شرایط خاص باعث کاهش پایداری، توسعه پذیری و انعطاف پذیری نرم افزار شوند. مهم ترین یافته های این بررسی در جدول زیر ارائه شده است.

شناسه یافته	ماژول مربوطه	شرح مشکل کشف شده	درجه اهمیت	اقدام اصلاحی (اصلاح کد)
CR-01	rule_engine.py	ریسک خطای مقایسه در صورت برگشت مقدار None از سنسور	بحرانی (High)	اضافه کردن شرط محافظ (Guard Clause) برای خروج امن
CR-02	logger.py	ثابت بودن نام فایل لاگ در کل برنامه	متوسط	تزریق نام فایل به عنوان پارامتر ورودی برای ماژولار شدن
CR-03	user.py	چک کردن مستقیم رشته Admin که توسعه پذیری نقش ها را ضعیف می کند	کم (Low)	آماده سازی متد برای پشتیبانی از لیست مجاز (Allow-list)

۳-۵. اصلاح کدهای پروژه:

بر اساس نتایج حاصل از بررسی کد، اصلاحاتی در بخش‌های مختلف پروژه انجام شد تا پایداری، خوانایی و قابلیت توسعه سیستم افزایش یابد. مهم‌ترین تغییرات اعمال‌شده در ادامه آورده شده است.

اصلاح ماژول موتور قوانین:

در نسخه اولیه، متد `evaluate` بدون بررسی مقدار دریافتی از سنسور، عملیات مقایسه دما را انجام می‌داد. در صورتی که سنسور مقدار `None` برمی‌گرداند، این موضوع می‌توانست باعث ایجاد خطا و توقف اجرای موتور قوانین شود. برای رفع این مشکل، یک شرط محافظ (Guard Clause) به ابتدای متد اضافه شد تا در صورت دریافت مقدار نامعتبر، سیستم بدون ایجاد خطا وضعیت را مدیریت کرده و اجرای برنامه ادامه پیدا کند.

```
C: > Users > SETUP CO > Desktop > testpythonfile.py
1 class RuleEngine:
2     def __init__(self, sensor, device, config):
3         self.sensor = sensor
4         self.device = device
5         self.config = config
6
7     def evaluate(self, simulated_temp):
8         if simulated_temp is None:
9             return "ALERT_HANDLED"
10
11         if simulated_temp > self.config.target_temp:
12             self.device.turn_on()
13             return "COOLER_ON"
14         else:
15             self.device.turn_off()
16             return "COOLER_OFF"
```

اصلاح ماژول ثبت وقایع:

در نسخه اولیه، نام فایل ثبت گزارش‌ها به صورت ثابت در برنامه تعریف شده بود. این موضوع باعث کاهش انعطاف‌پذیری سیستم در محیط‌های مختلف می‌شد. برای رفع این محدودیت، مسیر فایل به عنوان پارامتر ورودی در نظر گرفته شد تا در صورت نیاز بتوان فایل‌های متفاوتی برای ثبت گزارش‌ها استفاده کرد. همچنین مدیریت خطا هنگام دسترسی به فایل نیز به کد اضافه شد.

```
1 import datetime
2
3 class Logger:
4     @staticmethod
5     def log(message, file_path="system_log.txt"):
6         timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
7         with open(file_path, "a", encoding="utf-8") as file:
8             file.write(f"[{timestamp}] {message}\n")
9
```

در نهایت...:

فاز هفتم پروژه با هدف ارزیابی نهایی کیفیت نرم افزار، اعتبارسنجی عملکرد بخش های مختلف سیستم و افزایش قابلیت اطمینان آن انجام شد. در این مرحله، ابتدا طرح آزمون (Test Plan) تدوین شد و سپس مجموعه ای از آزمون های واحد برای بررسی عملکرد ماژول های اصلی سیستم طراحی و اجرا گردید. نتایج حاصل از اجرای این آزمون ها نشان داد که بخش عمده ای از منطق برنامه مطابق انتظار عمل می کند و خطاهای احتمالی به درستی مدیریت می شوند. همچنین گزارش پوشش آزمون، میزان مناسبی از پوشش کدهای پروژه را نشان داد و بیانگر آن بود که بیشتر بخش های مهم برنامه تحت آزمون قرار گرفته اند. در ادامه، با اجرای فرآیند بررسی کد (Code Review)، نقاط ضعف ساختاری و فنی موجود در نسخه اولیه شناسایی و اصلاح شدند. این اصلاحات موجب افزایش پایداری سیستم، بهبود خوانایی کد، افزایش انعطاف پذیری و آماده تر شدن پروژه برای توسعه های بعدی شد. در مجموع، انجام فعالیت های این فاز باعث شد نسخه نهایی سیستم مدیریت هوشمند انرژی ساختمان ها از نظر کیفیت نرم افزار، قابلیت نگهداری و پایداری عملکرد در وضعیت مطلوب تری قرار گیرد و مستندات پروژه نیز برای ارائه و تحویل نهایی تکمیل شوند.